

实验三：插值和逼近

实验性质

本实验为验证性和设计性实验。

实验目标

1. 掌握用于插值的各类算法
2. 掌握用于逼近的最小二乘法
3. 生成实验用的数据和函数
4. 学会使用插值和逼近方法求解实际问题

先决条件

具有 Python 编程的基本知识，了解与插值和逼近、线性代数有关的数学概念

实验材料

- 一台带有 Python 解释器的计算机
- 可以运行 Jupyter Lab 的实验环境

实验过程说明

插值和逼近实验

生成实验数据

下面的示例代码可以从你的学号生成用来练习用的数据点。

```
1 import random
2 import hashlib
3
4 def generate_data_points(school_number):
5     x_coordinates = list(range(8))
6     school_number_str = str(school_number)
7     hash_object = hashlib.sha256(school_number_str.encode('utf-8'))
8     hash_value = int(hash_object.hexdigest(), 16)
9     random.seed(hash_value)
10    y_coordinates = [random.uniform(-5, 5) for _ in range(8)]
11    return x_coordinates, y_coordinates
```

测试用例

```
1 school_number = 22030330 # 请替换成你的学号
2 data_points = generate_data_points(school_number)
```

插值

Lagrange 插值

Lagrange 插值使用 $x_1, x_2, \dots, x_n$ 一共 $(n + 1)$ 个点的信息。所构造的基函数形如

$$L_k(x) = \prod_{i=0, i \neq k}^n \frac{(x - x_i)}{(x_k - x_i)}$$

它满足 $L_k(x_i) = \begin{cases} 1 & \text{当 } i = k, \\ 0 & \text{当 } i \neq k. \end{cases} \quad i = 0, 1, \dots, n$

则 Lagrange 插值多项式为 $P_n(x) = \sum_{k=0}^n f(x_k) L_k(x)$

我们使用 Python 的函数式编程来处理这个问题，示例代码如下：

```
1  import operator
2  from functools import reduce
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6  def lagrange_basis(xs, i):
7      '''
8          在点 x_i 计算相应的 Lagrange 基
9      '''
10     n = len(xs)
11     assert i in range(n), '所给下标超出范围'
12
13     # 使用高阶函数
14     def _basis(x):
15         terms = [(x - xs[k]) / (xs[i] - xs[k]) for k in range(n) if k != i]
16         return reduce(operator.mul, terms, 1)
17     return _basis
18
19 def lagrange_interpolate(xs, ys):
20     '''
21         基函数的线性组合
22     '''
23     n = len(xs)
24     assert len(xs) == len(ys), '所给节点长度不匹配'
25
26     # 使用高阶函数
27     def _func(x):
28         return sum(lagrange_basis(xs, i)(x) * ys[i] for i in range(n))
29
30     return _func
31
32 # 示例插值节点
33 xvalues, yvalues = data_points
34
35 # 创建插值函数
36 ifunc = lagrange_interpolate(xvalues, yvalues)
```

```

37
38 xs = np.linspace(xvalues[0], xvalues[-1], 100)
39 ys = [ifunc(x) for x in xs]
40
41 plt.plot(xs, ys)
42 plt.plot(xvalues, yvalues, 'k.')
43 plt.show()

```

通过运行代码，可以直观地查看利用 Lagrange 插值的效果。

Newton 差商

$P_n(x)$ 写为 Newton 差商形式 $P_n(x) = f[x_0] + \sum_{k=1}^n f[x_0, x_1, \dots, x_k](x - x_0) \dots (x - x_{k-1})$

需要计算差商系数表

记 $F_{i,j} = f[x_{i-j}, \dots, x_i]$. 差商系数 $F_{0,0}, F_{1,1}, \dots, F_{n,n}$ 满足 $P_n(x) = F_{0,0} + \sum_{i=1}^n F_{i,i} \prod_{j=0}^{i-1} (x - x_j)$.

示例代码如下：

```

1  def newton_interpolate(xs, ys):
2      '''
3      生成一个函数，用于计算基于给定数据点的牛顿插值多项式的值。
4      '''
5
6      def _divided_difference(xs, ys):
7          # 请您实现
8
9          # 计算差商表并获取系数
10         coef = _divided_difference(xs, ys)
11
12         # 返回计算插值的函数
13         def _interpolated_function(x):
14             total = coef[0]
15             product_term = 1
16             for i in range(1, len(xs)):
17                 product_term *= (x - xs[i-1])
18                 total += coef[i] * product_term
19             return total
20
21         return _interpolated_function
22
23     # 示例插值节点
24     xvalues, yvalues = data_points
25
26     # 创建插值函数
27     ifunc = newton_interpolate(xvalues, yvalues)
28
29     xs = np.linspace(xvalues[0], xvalues[-1], 100)
30     ys = [ifunc(x) for x in xs]
31
32     plt.plot(xs, ys)
33     plt.plot(xvalues, yvalues, 'k.')

```

实验要求

根据课件提供的算法伪代码，使用 Python 语言编写示例函数 `newton_interpolate` 中的一个计算差商表的辅助函数 `_divided_difference`。完成这段代码之后，请执行查看结果。如果顺利的话，应该得到与 Lagrange 插值类似的图形。

三次样条

三次样条插值是一种平滑的插值方法，可以使用 Python 的 `scipy` 库中的 `interp1d` 函数实现。该方法在给定数据点之间创建一个三次多项式，从而生成一个平滑的曲线。

以下是示例代码

```
1  from scipy.interpolate import interp1d
2
3  # 示例插值节点
4  xvalues, yvalues = data_points
5
6  # 创建插值函数
7  ifunc = interp1d(xvalues, yvalues, kind='cubic')
8
9  xs = np.linspace(xvalues[0], xvalues[-1], 100)
10 ys = [ifunc(x) for x in xs]
11
12 plt.plot(xs, ys)
13 plt.plot(xvalues, yvalues, 'k.')
14 plt.show()
```

三次样条的插值结果看上去和数据点的局部趋势更加“熨帖”。

实验要求

保时捷 911 汽车顶盖的侧面流线形描点所得数据为

```
1  data = [
2      [ 0.0, 0.0],
3      [ 3.5, 2.0],
4      [ 5.9, 3.1],
5      [13.2, 4.2],
6      [15.4, 5.0],
7      [20.9, 7.6],
8      [24.5, 8.3],
9      [31.3, 7.8],
10     [42.2, 4.8],
11     [45.8, 3.4],
12     [46.5, 2.2]
13 ]
```

请使用三次样条插值并绘制保时捷 911 的顶盖曲线。

提示：在 Matplotlib 中，使用 `set_aspect` 可以保持不同的坐标轴的刻度间隔相等。从而可以绘制更逼真的保时捷顶盖。

逼近

对数据点使用插值得到的函数给出的信息是冗余的。最佳物理图景应该符合 Occam 剃刀原则，应该去找尽可能简约的描述。尽管逼近所用的函数不如插值函数精确，但能最佳逼近问题的解。

最小二乘法

最小二乘法是一种数学优化技术，它通过最小化误差的平方和来找到最佳函数匹配数据。最小二乘多项式逼近在很多领域都有应用，如物理、工程技术、经济学等。它用于曲线拟合，预测分析，信号处理和控制系统等多种问题。

在函数逼近问题中，最小二乘法可以用来确定一组基函数的最佳线性组合来逼近一组数据点。需要确定多项式的系数，使得该多项式最好地符合一组给定的数据点。例如，给定数据点 (x_i, y_i) ，我们可能希望找到一个多项式 $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ 来近似这些点。目标是选择系数 $a_0, a_1, \dots, a_n$ 来最小化误差函数：

$$E(a_0, a_1, \dots, a_n) = \sum_{i=1}^m (y_i - p(x_i))^2$$

其中 m 是数据点的数量。

为了找到最小化误差函数的系数，可以通过解正规方程来实现。正规方程是通过求解导数（梯度）等于零的条件来得到的。对于多项式逼近问题，可以将问题表述为线性代数形式，其中系数向量 β 需要满足：

$$\mathbf{A}^\top \mathbf{A} \beta = \mathbf{A}^\top \mathbf{y}$$

这里， $\mathbf{A}$ 是设计矩阵，每一行包含了对应 x_i 的多项式项的值， $\mathbf{y}$ 是观测值的向量。求解这个方程组将给出最小二乘意义上的“最佳”系数。

实验要求

已知中国人口数据如下表

年份（年）	1953	1964	1982	1990	2000
人口（亿）	5.82	6.95	10.08	11.34	12.66

编写 Python 程序，使用最小二乘法拟合这组数据，并据此预测 2010 年的人口。

提示：形如 $f(x) \approx \frac{a + bx}{1 + cx}$ 的函数，可以先变形为 $f(x)(1 + cx) \approx a + bx$ 再处理

我们把方程变形为 $a + bx - cx f(x) \approx f(x)$ ，这样就成为一个线性最小二乘问题。把 a 、 b 和 c 当作是未知数，建立线性方程组进行求解。

- 参考上述描述，使用 Python 语言编程计算。注意，需要从所给的数据中得到拟合函数 `afunc`，它接受实际的年份数值为参数。
- 试着对输入年份做平移，把所有年份减去首年的数值（1953）。得到的数值较小，与人口数值相仿。用这样的输入数据再做最小二乘拟合并预测 2010 年的人口。请和第一题的结果作比较。有没有差异，为什么？

撰写实验报告

1. 总结该实验，包括实验目标、实验方法和实验过程。
2. 描述实验实施的具体情况和结果，包括遇到的问题和挑战，已经如何解决问题的思路。
3. 分析实验结果，并根据所使用的数据和方法得出结论。
4. 将实验报告誊写到报告书模版文档。